

```

        return (x, y) # Return the starting coordinates.

def solveMaze(maze, x=None, y=None, visited=None):
    if x == None or y == None:
        x, y = findStart(maze)
        maze[y][x] = EMPTY # Get rid of the 'S' from the maze.
    if visited == None:
        ❶ visited = [] # Create a new list of visited points.

    if maze[y][x] == EXIT:
        return True # Found the exit, return True.

    maze[y][x] = PATH # Mark the path in the maze.
    ❷ visited.append(str(x) + ',' + str(y))
    ❸ #printMaze(maze) # Uncomment to view each forward step.

    # Explore the north neighboring point:
    if y + 1 < HEIGHT and maze[y + 1][x] in (EMPTY, EXIT) and \
    str(x) + ',' + str(y + 1) not in visited:
        # RECURSIVE CASE
        if solveMaze(maze, x, y + 1, visited):
            return True # BASE CASE
    # Explore the south neighboring point:
    if y - 1 >= 0 and maze[y - 1][x] in (EMPTY, EXIT) and \
    str(x) + ',' + str(y - 1) not in visited:
        # RECURSIVE CASE
        if solveMaze(maze, x, y - 1, visited):
            return True # BASE CASE
    # Explore the east neighboring point:
    if x + 1 < WIDTH and maze[y][x + 1] in (EMPTY, EXIT) and \
    str(x + 1) + ',' + str(y) not in visited:
        # RECURSIVE CASE
        if solveMaze(maze, x + 1, y, visited):
            return True # BASE CASE
    # Explore the west neighboring point:
    if x - 1 >= 0 and maze[y][x - 1] in (EMPTY, EXIT) and \
    str(x - 1) + ',' + str(y) not in visited:
        # RECURSIVE CASE
        if solveMaze(maze, x - 1, y, visited):
            return True # BASE CASE

    maze[y][x] = EMPTY # Reset the empty space.
    ❹ #printMaze(maze) # Uncomment to view each backtrack step.

    return False # BASE CASE

```